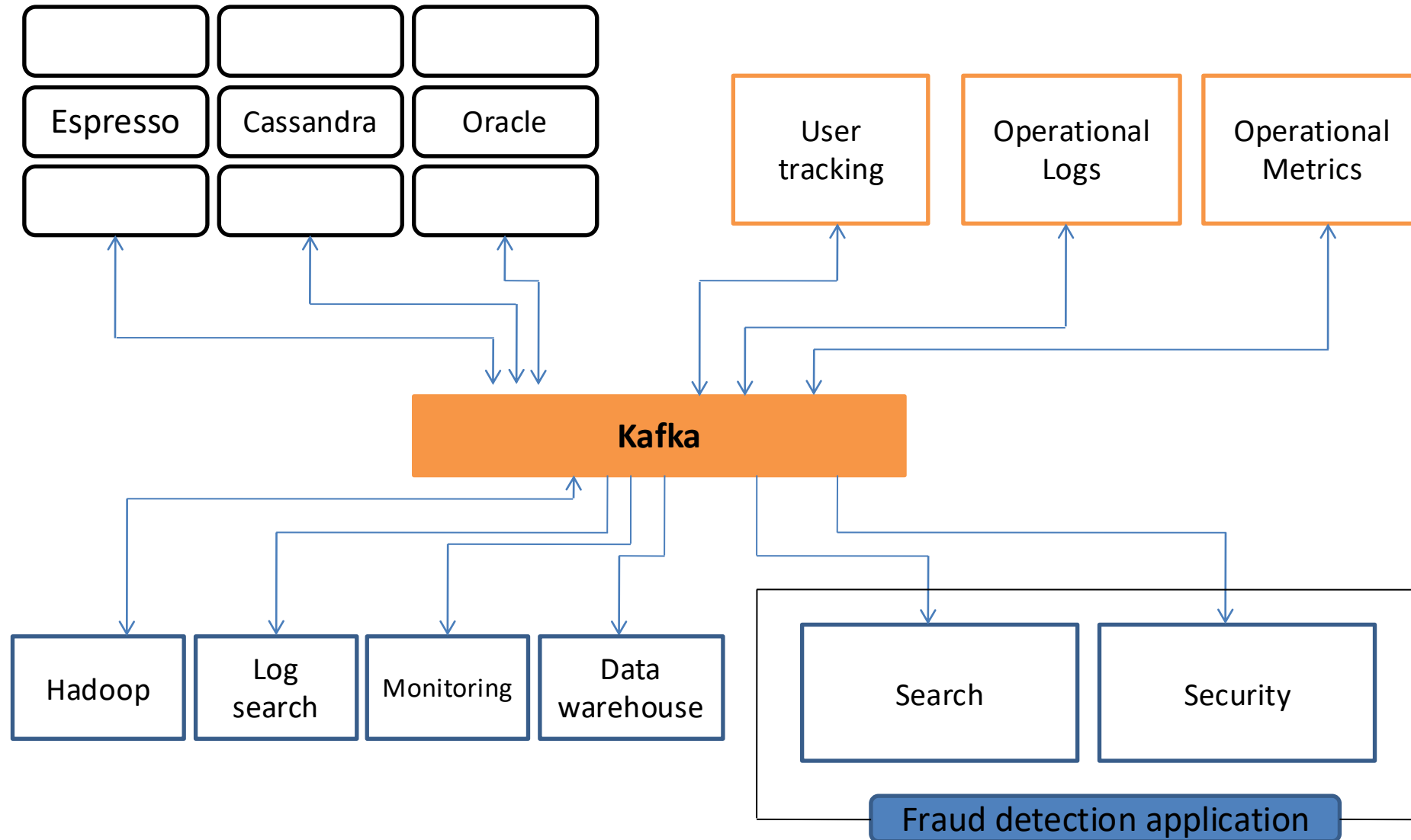


Kafka Connect

Simplified, scalable data import/export for Kafka

Central Hub with Kafka

Tos



Kafka Connect is a distributed, scalable, fault-tolerant service designed to reliably stream data between Kafka and other data systems.

Data is produced from a source and consumed to a sink.

Example use cases:

- Publishing an SQL Tables (or an entire SQL database) into Kafka
- Consuming Kafka topics into HDFS for batch processing
- Consuming Kafka topics into Elasticsearch for secondary indexing
- Integrating legacy systems with modern Kafka framework
- ... many others

Kafka producers and consumers are simple in theory, yet ungainly in practice

Many common requirements

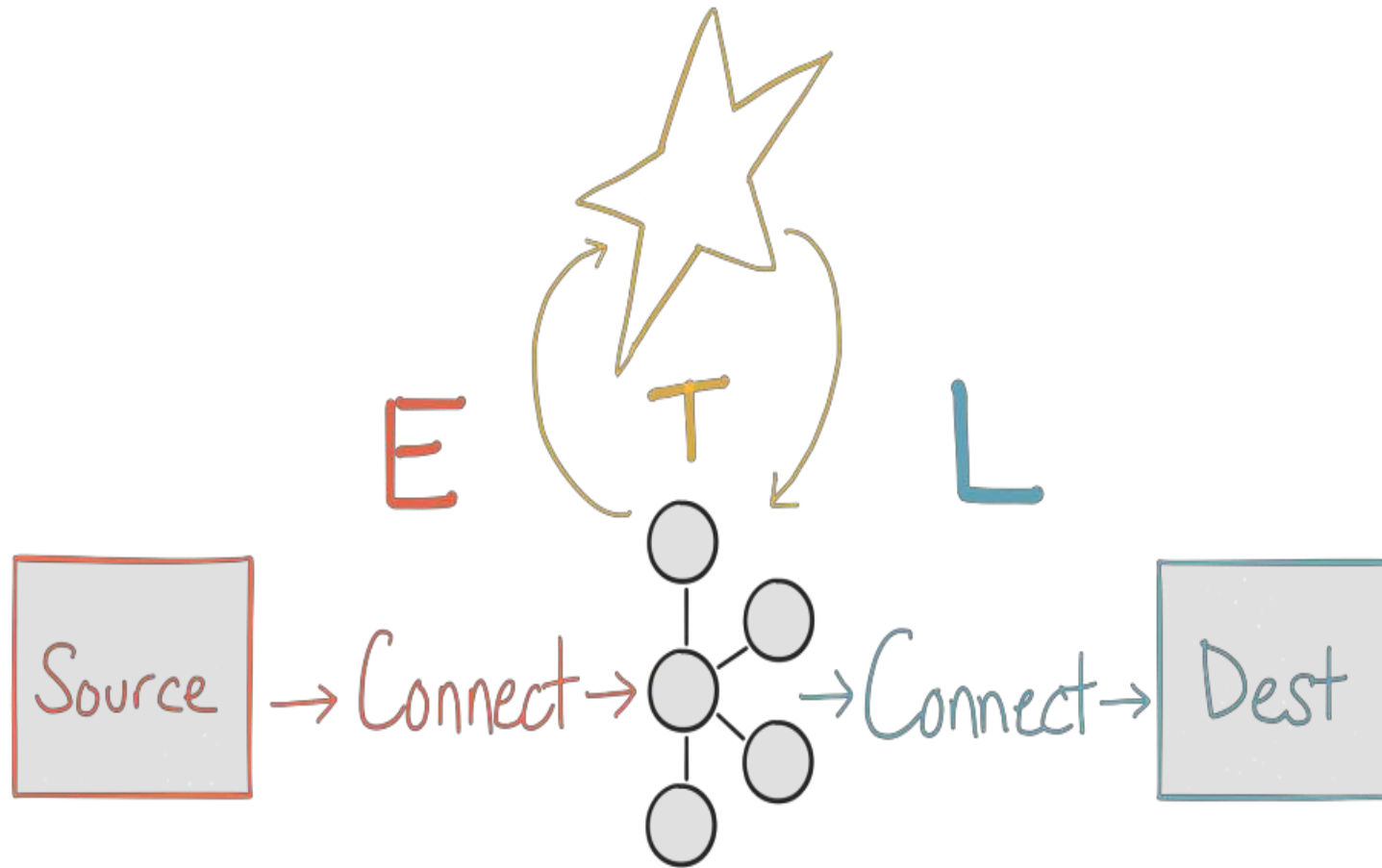
- Data conversion (serialization)
- Parallelism / scaling
- Load balancing
- Fault tolerance / fail-over
- General management

A few data-specific requirements

- Move data to/from sink/source
- Support relevant delivery semantics

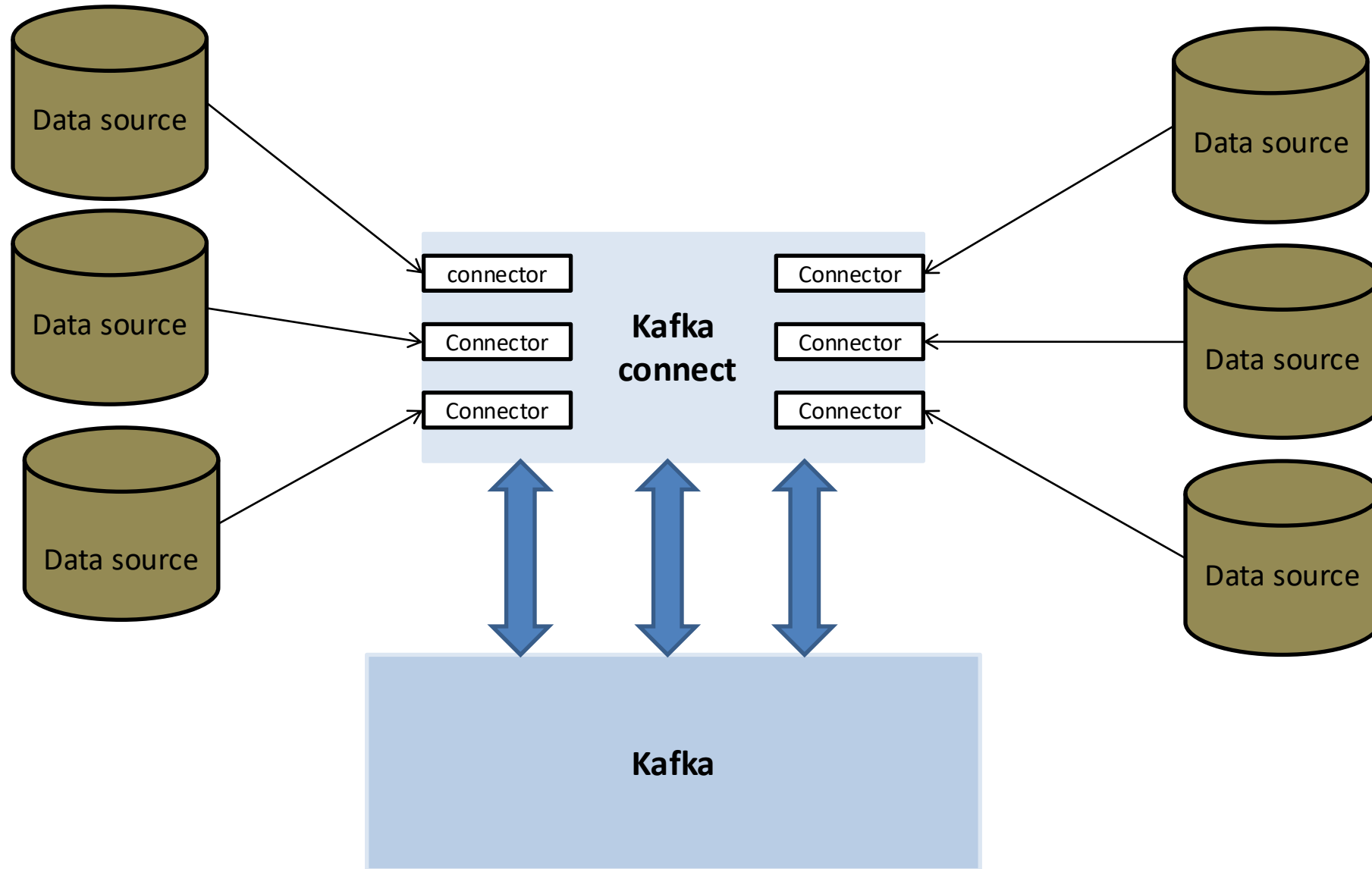
Kafka Connect : Separation of Concerns

Tos



Kafka Connect : Source & Sink Connectors

Tos



Connectors and Tasks



JDBC SOURCE

S3 SINK

WORKER

S3 TASK #1

JDBC TASK #1

JDBC TASK #2

Connect workers can run in two modes: standalone or distributed

Standalone mode runs in a single process -- ideal for testing and development, and consumer/producer use cases that need not be distributed (for example, tailing a log file).

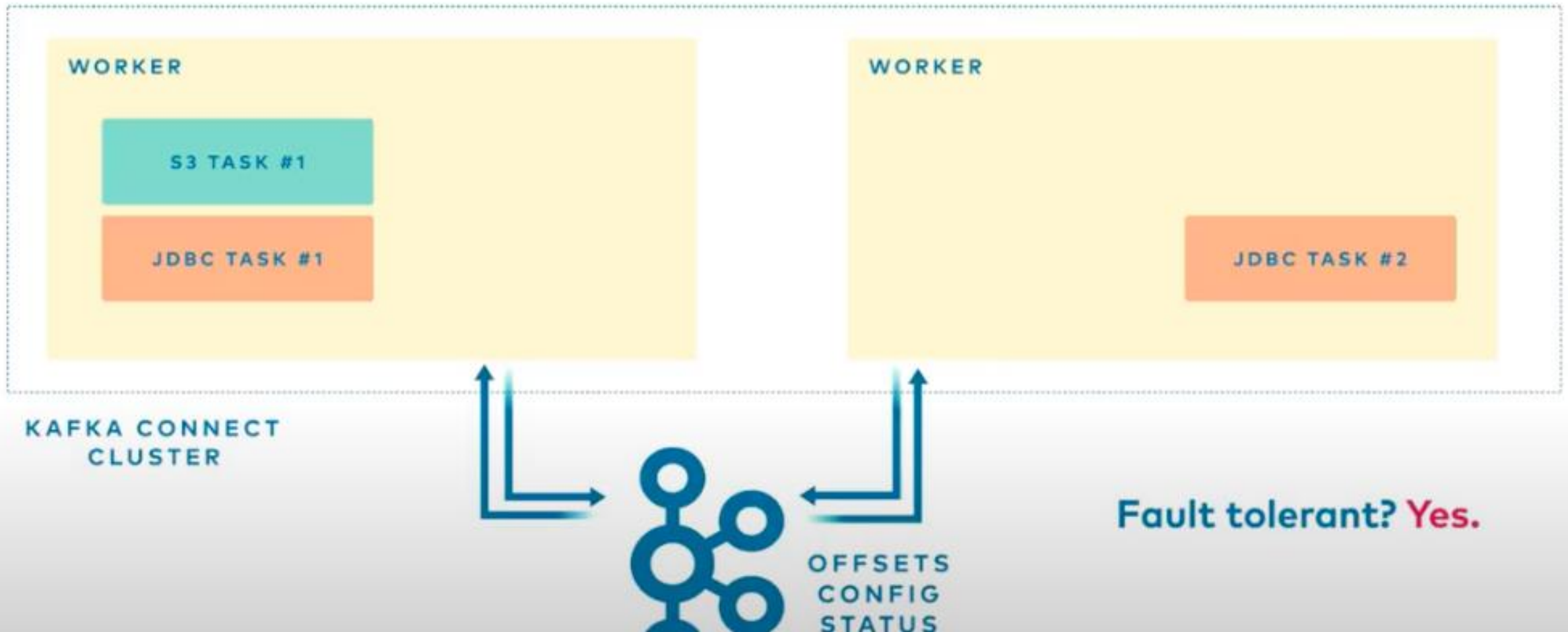
Distributed mode runs in multiple processes on the same machine or spread across multiple machines. Distributed mode is fault tolerant and scalable (thanks to Kafka Consumer Group support).

Kafka Connect Standalone Worker



Fault-tolerant? **No.**

Scaling the Distributed Worker



Pluggable API to convert data between native formats and Kafka

- **Source connectors:** converters are invoked after the data has been fetched from the source and before it is published to Kafka
- **Sink connectors:** converters are invoked after the data has been consumed from Kafka and before it is stored to the sink
- Converters are applied to both key and value data

Apache Kafka ships with JSONConverter

Confluent Platform adds AvroConverter

- AvroConverter allows integration with Confluent Schema Registry to track topic schemas and ensure compatible schema evolution.

- ❖ In **standalone** mode, Connector configuration is stored in a file and specified as a command-line argument.
- ❖ In **distributed** mode, Connector configuration is set via the REST API, in the JSON payload.
- ❖ Connect configuration options are as follows:

Parameter	Description
Name	Connector's unique name.
Connector.class	Connector's Java class
Tasks.max	Maximum tasks to create. The Connector may create fewer if it cannot achieve this level of parallelism
Topics(sink connectors only)	List of input topics(to consume from)

These parameters define the worker connection to the Kafka cluster. See docs.confluent.io for a comprehensive list.

Parameter	Description
bootstrap.servers	A list of host/port pairs to use for establishing the initial connection to the Kafka Cluster.
Key.converter	Converter class for key connect data.
value.converter	Converter class for value connect data

- Standalone mode

Parameter	Description
offset.storage.file.filename	The file to store Connector offsets in.

- Distributed mode

Parameter	Description
group.id	A unique string that identifies the Connect cluster group this worker belongs to.
config.storage.topic	The topic to store Connector and task configuration data in. This must be the same for all workers with the same group.id.
offset.storage.topic	The topic to store offset data for Connectors in. This must be the same for all workers with the same group.id.
session.timeout.ms	The timeout used to detect failures when using Kafka's group management facilities.
heartbeat.interval.ms	The expected time between heartbeats to the group coordinator when using Kafka's group management facilities. Must be smaller than session.timeout.ms.

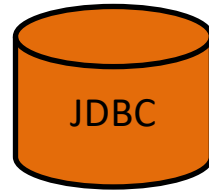
- Starting Connect in distributed mode involves starting connect on each worker node with the following:

connect-distributed worker.properties

- Connectors are then added, modified, and deleted via a REST API. Configuration will be covered later.

Method	Path	Description
GET	/connectors	Get a list of active connectors.
POST	/connectors	Create a new connector
PUT	/connectors/(string:name)/config	Create a new connector, or update the configuration of an existing connector
GET	/connectors/(string: name)/config	Get configuration info for a connector.
GET	/connectors/(string: name)/tasks/<tasks-id>/status	Retrieve details for specific tasks
DELETE	/connectors/(string:name)	Delete a configured connector from the worker pool

- ❖ Confluent-supported connectors (included in Confluent Platform)



- ❖ Community-written connectors (just a sampling)



Lab : Kafka Connector (JDBC) - 150 Minutes